

AI-Based Iris Recognition with Liveness Detection for Biometric Authentication

AI Project Proposal | Biometric Security | Deep Learning | Computer Vision

4 PROBLEM STATEMENT

Traditional authentication methods (passwords, PINs, tokens) are vulnerable to theft, phishing, and spoofing attacks. Existing biometric systems using fingerprint or face ID can be defeated by photos or 3-D replicas. A robust system is needed that can verify a user's identity using the unique patterns of the iris **and** confirm the eye is from a live person — not a printed image or artificial eye — in real time.

5 HYPOTHESIS

If I apply a dual-branch deep learning model combining a CNN-based iris texture classifier with a liveness detection module (using blink/pupil-dilation cues), **then** the system will authenticate genuine users with $\geq 95\%$ accuracy while rejecting spoofing attempts with $\geq 90\%$ precision, **because** the iris has ~ 240 degrees of freedom that are unique even among twins, and dynamic liveness signals cannot be replicated by static artefacts.

9 SUCCESS METRICS

Iris Recognition Accuracy	$\geq 95\%$
Liveness Detection Precision	$\geq 90\%$
False Acceptance Rate (FAR)	$< 1\%$
False Rejection Rate (FRR)	$< 5\%$
Authentication Response Time	< 2 seconds
Anti-Spoofing (EER)	$< 5\%$

8 DATASET SOURCES

- **CASIA-Iris-V4** — large multi-class iris DB
- **UBIRIS v2** — noisy real-world conditions
- **LivDet-Iris 2020** — liveness/spoof samples
- **Synthetic augmentation** — GAN-generated variants to balance classes

10 RISKS & MITIGATIONS

- **Limited spoofing data** → augment with GANs & transfer learning
- **Model overfitting** → dropout, batch-norm, early stopping
- **Poor generalisation** → cross-dataset validation
- **Hardware** → Google Colab Pro / cloud GPU

6 METHODOLOGY

1. Data Collection

Gather labeled iris images (real vs. spoofed) from public datasets.

2. Preprocessing

Segment iris using Hough transform, normalize to 8×64 strip, apply Gabor filters, augment data.

3. Model Design

Branch A — IrisNet CNN for texture encoding; Branch B — LSTM on blink/pupil sequences for liveness.

4. Training

Train jointly with combined loss (CrossEntropy + BCE). Use 80/10/10 split.

5. Evaluation

Test on held-out set; report FAR, FRR, EER, accuracy, latency.

6. Deployment

Wrap in REST API; integrate with webcam or IR camera front-end for live demo.

7 TOOLS & FRAMEWORKS

Python 3.10	PyTorch	OpenCV	Scikit-learn
HuggingFace	NumPy / Pandas	FastAPI	Matplotlib

PROJECT SNAPSHOT

Domain	Biometric Security / CV
Approach	Dual-branch CNN + LSTM
Input	Iris image / video stream
Output	Auth: PASS / FAIL + score
Timeline	12–16 weeks